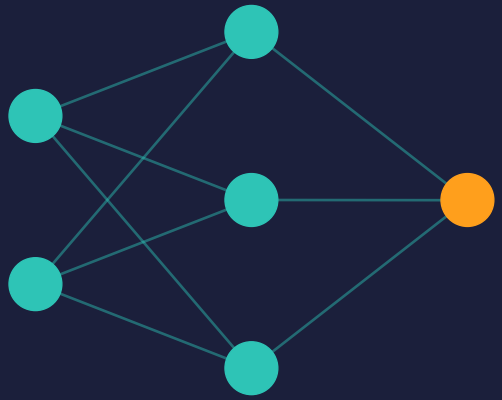
دوره یادگیری عمیق و شبکه‌های عصبی



جلسه ۹

یادگیری انتقالی و Fine-tuning

Transfer Learning: استفاده از مدل‌های ازپیش‌آموزش‌دیده برای مسائل با داده کم

کیوان محمودی

در این جلسه

۱ چرا یادگیری انتقالی؟

۲ مدل‌های ازپیش‌آموزش‌دیده در torchvision

۳ دیتاست Flowers-102

۴ Mixed Precision

۵ آموزش از صفر

۶ Feature Extraction

۷ Fine-tuning با نرخ یادگیری متفاوت

۸ راهنمای عملی

ایده یادگیری انتقالی

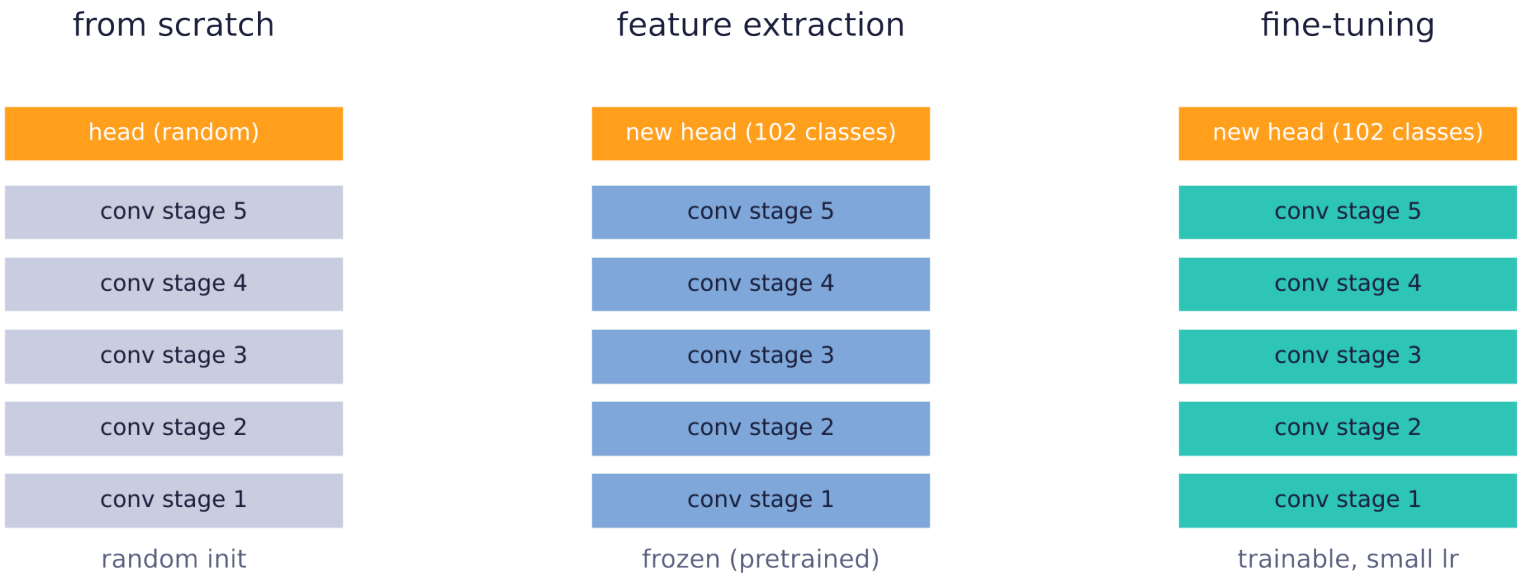
مسئله

- آموزش ResNet روی ImageNet: ۱/۲ میلیون تصویر و ساعت‌ها محاسبه
- در پروژه واقعی: چند صد یا چند هزار تصویر
- آموزش از صفر با داده کم: Overfitting شدید

راه‌حل

- مدل آموزش‌دیده روی داده بزرگ = نقطه شروع
- ویژگی‌های لایه‌های اول (لبه، رنگ، بافت) برای همه مسائل مفیدند.
- فقط لایه‌های آخر مختص مسئله اصلی‌اند.
- همین ایده در NLP: BERT (جلسه ۱۴)

سه راهبرد



Feature Extraction: داده بسیار کم | Fine-tuning: داده متوسط یا مسئله متفاوت‌تر | از صفر: داده بسیار زیاد

بارگذاری مدل و پیش‌پردازش متناظر آن

```
from torchvision import models

weights = models.ResNet18_Weights.IMAGENET1K_V1
model = models.resnet18(weights=weights)
preprocess = weights.transforms()   # resize 256, crop 224, normalize

weights.meta['categories'][:3]       # ImageNet class names
model.fc                             # Linear(512 -> 1000)
```

- هر وزن پیش‌پردازش دقیق زمان آموزش خودش را دارد.
- استفاده از پیش‌پردازش متفاوت یک خطای رایج است.
- میانگین و انحراف معیار ImageNet را فراموش نکنید.

دیتاست Oxford Flowers-102

- ۱۰۲ گونه گل (Nilsback & Zisserman, 2008)
- فقط ۱۰ تصویر آموزشی برای هر کلاس: ۱۰۲۰ تصویر
- ۱۰۲۰ تصویر اعتبارسنجی و ۶۱۴۹ تصویر آزمون
- شرایط داده کم و کلاس زیاد: جایی که یادگیری انتقالی می‌درخشد
- Augmentation آموزش: RandomResizedCrop و قرینه افقی
- Label Smoothing برابر ۰/۱ برای جلوگیری از اطمینان بیش از حد

Mixed Precision: سریع‌تر و کم‌حافظه‌تر

- بیشتر محاسبات با float16 روی Tensor Coreها
- وزن‌ها با دقت float32 نگه داشته می‌شوند.
- GradScaler گرادیان‌های کوچک را از صفر شدن حفظ می‌کند.

```
scaler = torch.amp.GradScaler()

for X, y in loader:
    with torch.autocast(device_type='cuda', dtype=torch.float16):
        logits = model(X)
        loss = criterion(logits, y)
    optimizer.zero_grad()
    scaler.scale(loss).backward()   # avoid float16 underflow
    scaler.step(optimizer)
    scaler.update()
```

Feature Extraction: منجمد کردن Backbone

```
model = models.resnet18(weights=weights)
for p in model.parameters():
    p.requires_grad = False                    # freeze
model.fc = nn.Linear(model.fc.in_features, 102)  # new head

optimizer = torch.optim.AdamW(model.fc.parameters(), lr=1e-3)
# trainable: 52,326 of 11.2 M parameters (0.47%)
```

- فقط لایه جدید آموزش می‌بیند؛ بسیار سریع
- Backpropagation فقط تا لایه آخر
- نکته: آمار BatchNorm منجمد را در حالت eval نگه دارید.

Fine-tuning با نرخ یادگیری متفاوت

```
model = models.resnet18(weights=weights)
model.fc = nn.Linear(model.fc.in_features, 102)

backbone = [p for n, p in model.named_parameters()
            if not n.startswith('fc.')]
optimizer = torch.optim.AdamW([
    {'params': backbone,                'lr': 1e-4},
    {'params': model.fc.parameters(),   'lr': 1e-3},
], weight_decay=1e-4)
```

- Parameter Groups: نرخ یادگیری جدا برای هر بخش
- نرخ کوچک برای لایه‌های پیش‌آموزش‌دیده تا دانششان از بین نرود
- نرخ بزرگ‌تر برای Head که از صفر شروع می‌کند

راهنمای عملی

۱ پیش‌پردازش

همیشه همان پیش‌پردازش مدل پیش‌آموزش‌دیده را استفاده کنید.

۲ شروع

با Feature Extraction شروع کنید؛ سریع است و خط مبنای خوبی می‌دهد.

۳ Fine-tuning

نرخ Backbone را ۱۰ تا ۱۰۰ برابر کوچک‌تر بگیرید یا Gradual Unfreezing.

۴ منابع

torchvision.models، کتابخانه timm و Hugging Face Hub

جمع‌بندی

- یادگیری انتقالی دانش مدل آموزش‌دیده روی داده بزرگ را به مسئله با داده کم منتقل می‌کند.
- Feature Extraction: منجمد کردن Backbone و آموزش Head جدید
- Fine-tuning: آموزش همه لایه‌ها با نرخ یادگیری کوچک
- Parameter Groups برای نرخ یادگیری متفاوت
- Mixed Precision با autocast و GradScaler
- Label Smoothing منظم‌سازی ساده برای داده کم

تمرین

۱ به‌جای ResNet-18 از efficientnet_b0 و resnet50 استفاده کنید و نتایج را مقایسه کنید.

۲ Gradual Unfreezing را پیاده کنید: ابتدا fc، سپس layer4، سپس همه لایه‌ها

۳ در Feature Extraction لایه‌های BatchNorm را در حالت eval نگه دارید و مقایسه کنید.

۴ با ImageFolder و عکس‌های خودتان یک طبقه‌بند کوچک بسازید.